

January 2026: CSE310 – Compiler Sessional

Getting Started with ANTLR4

Prepared by – Md Irtiaz Kabir

July 2026

1 Introduction

ANTLR (ANother Tool for Language Recognition) is a robust parser generator designed for reading, processing, executing, or translating structured text or binary data. It is extensively used in the development of programming languages, tools, and frameworks. Given a grammar, ANTLR produces a parser capable of constructing and traversing parse trees.

2 Setting things up

This section outlines the installation process for Linux keeping details limited to much basic as possible. There are 2 steps to installing ANTLR for our assignment - the runtime and the tools.

2.1 ANTLR C++ runtime

When setting up ANTLR for C++ development, it's essential to understand the role of the ANTLR runtime. The runtime is a library that provides the necessary support for the parser and lexer code generated by ANTLR to function correctly within our C++ application. Including the runtime involves integrating the ANTLR C++ runtime library into our project. This library contains implementations of various components such as token streams, parsers, lexers, and tree walkers, which are crucial for the operation of the generated code. By incorporating the runtime, we ensure that our application has access to these components, enabling it to parse and process input as defined by our grammar.

2.1.1 Steps of including the ANTLR C++ runtime

The minimum C++ version to compile the ANTLR C++ runtime with is C++17 and the minimum CMake version required is CMake 2.8. While earlier versions of CMake (e.g., 2.8) might suffice, it is advisable to use a more recent version to ensure compatibility and access to the latest features. The default CMake version available through apt on Ubuntu may be outdated, potentially leading to compatibility issues. To install the latest version of CMake, follow these steps:

Remove existing installation This step ensures that any existing, potentially outdated, versions of CMake are removed from your system. First give

```
$ which cmake
```

- **If installed from source**

If this returns a path like `/usr/local/bin/cmake`, it indicates that CMake was installed manually and not managed by APT. Navigate to the directory where you built CMake and run:

```
$ sudo make uninstall
```

This will remove the installed files. If you no longer have the build directory, you can recreate it by downloading the same version of the source code used for installation, running `./bootstrap` or `./configure`, `make`, and then `sudo make uninstall`. [Follow the documentation properly and give `uninstall` instead of `install`]

- **If installed via `.sh` installer:**

If you used a script like `cmake-3.31.7-linux-x86_64.sh` to install CMake, it likely installed to `/usr/local`. To remove it. **BE AWARE ABOUT THE PATHS AND USING `rm -rf`**

```
$ sudo rm -rf /usr/local/bin/cmake
$ sudo rm -rf /usr/local/share/cmake*
$ sudo rm -rf /usr/local/doc/cmake*
$ sudo rm -rf /usr/local/man/man1/cmake*
```

- **If installed via Snap**

If you see path like `/snap/bin/cmake`, then it indicates that CMake was installed via Snap. To remove this Snap-installed version of CMake, run the following command:

```
$ sudo snap remove --purge cmake
```

Then verify removal with `which cmake`

- **If installed via apt**

If you see a path like `/usr/bin/cmake`, then it was probably installed using apt. To remove it –

```
$ sudo apt purge --auto-remove cmake
$ hash -r
```

Installation of CMake You can follow any of the following two for installing CMake 3.* (newer version (3.5 or higher) may have backward compatibility issues).

- **Add Kitware APT Repository**

Kitware provides up-to-date CMake packages. Adding their repository allows you to install the latest version directly. (Be aware about the line-break)

However, executing the attached `cmake-installation-script.sh` will also do. A good resource about installing the latest CMake – [Ask Ubuntu](#)

```
$ wget -O - https://apt.kitware.com/keys/kitware-archive-latest.asc 2>/dev/null | gpg --
  dearmor - | sudo tee /etc/apt/trusted.gpg.d/kitware.gpg >/dev/null
$ sudo apt-add-repository "deb https://apt.kitware.com/ubuntu/ $(lsb_release -cs) main"
$ sudo apt update
$ sudo apt install cmake=3.25.*
$ hash -r
```

You can simply give `sudo apt install cmake` without specifying any version.

- **Installing a specific version of CMake by compiling source**

Compiling CMake from source is an excellent option. This method involves downloading, building, and installing the source code manually. [LinuxCapable.com](#) provides detailed step. You may follow this article.

2.1.2 Installing the ANTLR4 C++ Runtime

The file `antlr4-runtime.h` is part of the ANTLR4 C++ runtime library. To obtain it, let's follow execute the commands one by one: (executing the attached `antlr-runtime-installation-script.sh` will also do)

```
$ git clone https://github.com/Irtiaz/antlr4.git
$ cd antlr4
$ cd runtime/Cpp
$ mkdir build && cd build
$ cmake ..
$ make
$ sudo make install
```

This will compile the runtime and install the headers and libraries on the system so that compiler can find `antlr4-runtime.h` when included. Many systems place headers in `/usr/local/include` or `/usr/include`. If the runtime is installed there, the compiler may find them by default. Running `find /usr/local/include -name "antlr4-runtime.h"`, the installation can be verified.

2.2 Installation of antlr4-tools

To play around with ANTLR without having to worry about installing it and the Java needed to execute it, use [antlr4-tools](#). The only requirement is Python3. But at first, we may require creating a virtual environment for installation using `pip`. The command to create and activate the environment

```
$ python3 -m venv <environment_name> # this creates virtual environment
$ source <environment_path>/bin/activate # for environment activation
```

Suppose you want to create an environment named `antlr4_venv` in `~/3-1/CSE310` directory. So, navigate to the directory and then execute the following commands.

```
$ python3 -m venv antlr4_venv # this creates virtual environment
$ source ./antlr4_venv/bin/activate # for environment activation
```

Now you should see the shell like this

```
(antlr4_venv) $
```

Now as you have the environment activated, use the following command which creates `antlr4` and `antlr4-parse` executables. (You can deactivate an environment by simply writing `deactivate` and then pressing enter)

```
$ pip install antlr4-tools
$ antlr4 -v 4.13.2
```

3 Building a project

3.1 Visualizing the parse tree

You can visualize the parse tree for a particular input, with the following command -

```
$ antlr4-parse -v 4.13.2 <grammar>.g4 <start_symbol> -gui
```

Replace `<grammar>` with the name of your grammar file, and `start_symbol` with the name of the start symbol of your grammar. Then provide the input from console (`stdin`). To terminate the input, press `Ctrl+D`

If you want to provide the input from a file (say `input.txt`), you can use this bash trick,

```
$ antlr4-parse -v 4.13.2 <grammar>.g4 <start_symbol> -gui < input.txt
```

3.2 Generating with Antlr4

To actually use the parse tree from code, we will need to generate visitor pattern code from Antlr4 grammar (`.g4`) files. We can use the following command -

```
$ antlr4 -v 4.13.2 <grammar>.g4 -Dlanguage=Cpp -no-listener -visitor
```

Replace `<grammar>` with the name of your grammar file.

3.3 Compiling the project

When compiling project (for example, using `g++`), we need to use the `-I` option to add the path to the directory that contains `antlr4-runtime`. Suppose we have a source file named `main.cpp`, which is designed to parse an input file specified through command- line arguments. To compile this file and produce an executable named `main.out`, one can use the following commands. These commands are provided for illustrative purposes only.

```
$ g++ -std=c++17 -I/path/to/antlr4-runtime/include -c ExprLexer.cpp ExprParser.cpp main.cpp
$ g++ -std=c++17 ExprLexer.o ExprParser.o main.o -L/path/to/antlr4-runtime/lib -lantlr4-runtime -o
  main.out -pthread
```

Or you can do this single line build script as well -

```
$ g++ -std=c++17 -I/usr/local/include/antlr4-runtime -L/usr/local/lib/ -pthread *.cpp -lantlr4-
  runtime -o main.out
```

Here we are assuming that the path to `antlr4-runtime` is `/usr/local/include/antlr4-runtime`. This produces `main.out` file which can be run using the following command -

```
$ LD_LIBRARY_PATH=/usr/local/lib ./main.out
```

3.4 Generation, building and running - all at once

The attached `run-script.sh` file does everything for you. It even runs the executable file with a command line argument (some `input.txt`).

Acknowledgment

Adapted from the original Getting Started guide by Nafis Tahmid, Lecturer, Department of CSE, BUET